EAT N' GO LIMITED

BUSINESS INTELLIGENCE ENGINEER TECHNICAL CASE STUDY

JSON Data Pipeline, Data Model and BI Engineering Assessment

Role	Business Intelligence Engineer
Assessment Type	Technical Case Study
Recommended Completion Time	4 Days
Data Format	JSON files
Submission Focus	JSON ingestion, parsing, modelling, data quality, reconciliation, automation, API readiness and documentation

Candidate should submit all required deliverables using the naming convention provided in this document.

Assessment Overview

This case study assesses the candidate's ability to build the technical foundation behind business intelligence reporting. The focus is not only on analysis, but on reliable ingestion, transformation, validation, data modelling and production readiness.

Section	Purpose
Business Context	Explains the reporting need and the role of the BI Engineer.
Data Files Provided	Lists the six JSON files supplied to candidates.
Candidate Instructions	Defines the environment, technical focus and submission expectations.
Case Study Questions	Provides detailed engineering tasks covering ingestion, modelling, data quality, reconciliation and automation.
Required Deliverables	Lists all materials candidates must submit.
Presentation Requirement	Explains the presentation format for shortlisted candidates.
Evaluation Criteria	Provides a 100-point scoring guide.
Submission Checklist	Confirms everything expected before submission.

1. Business Context

Eat N' Go operates quick-service restaurant stores across multiple Nigerian cities. The business receives data from multiple operational and third-party systems, including order platforms, delivery partners, customer databases, product master data and store systems.

Leadership requires a reliable and automated data foundation that can support reporting on sales performance, product mix, customer behaviour, channel performance, delivery SLA and operational decision-making.

You have been provided with six JSON files. Your task is to ingest, parse, clean, validate, transform and model the data into reporting-ready tables that can support downstream BI dashboards and analytics.

This assessment is focused on your ability to build the technical foundation behind Business Intelligence reporting.

2. Data Files Provided

You will receive the following six JSON files:

JSON File	Expected Use
customers.json	Customer master data and customer-level attributes.
deliveries.json	Delivery records and delivery SLA details.
order_items.json	Order line items and product-level transaction details.
orders.json	Order-level transactions, including store, customer, channel and revenue fields.
products.json	Product master data and category details.
stores.json	Store master data including location and operational attributes.

You are expected to inspect the JSON structure, understand the fields, identify relationships across the files and design a clean data pipeline for reporting.

3. Candidate Instructions

3.1 Environment Requirement

You are required to load and process the six JSON files using your preferred environment.

You may use any of the following:

- PostgreSQL
- MySQL
- SQL Server
- SQLite
- DuckDB
- BigQuery
- Snowflake
- Amazon Redshift
- Python with SQL
- Spark
- Databricks
- Any other SQL-supported or data engineering environment

Your solution should include SQL scripts. Python or notebook-based processing is allowed, but the final transformed data should be available in a structured SQL-ready format.

3.2 Required Technical Focus

Your work should demonstrate your ability to:

- Ingest JSON files into a raw data layer
- Parse JSON into structured staging tables
- Validate data completeness and quality
- Design fact and dimension tables
- Build reporting-ready views or marts
- Reconcile source data to reporting outputs
- Design a repeatable ingestion pipeline
- Explain how the solution would work in production
- Handle late-arriving, corrected or duplicated records
- Prepare the data for BI tools such as Power BI, Tableau or Looker

4. Case Study Objectives

1. Inspect and understand the six JSON files.
2. Load the JSON files into a raw or staging layer.
3. Parse and structure the JSON data into relational tables.
4. Design a clean analytical data model.
5. Build data quality and reconciliation checks.
6. Create reporting-ready views or tables.
7. Design an automated daily pipeline.
8. Explain how the same approach would support future API integrations.
9. Provide technical documentation for your solution.

5. Case Study Questions

Question 1: JSON Source Review and Profiling

Review the six JSON files and provide a source profiling summary.

Your answer should cover:

- Record count for each JSON file
- List of key fields in each file
- Data type assumptions
- Primary key candidates
- Foreign key relationships
- Null or missing values
- Duplicate records
- Invalid values
- Date range covered by the data
- Any schema inconsistencies observed

Expected output:

- SQL, Python or notebook profiling queries
- Summary table of profiling results
- Short explanation of data issues identified

Question 2: Raw and Staging Layer Design

Design how the JSON files should be stored before and after parsing.

Your answer should cover:

- How the raw JSON files will be stored
- Whether raw records should be preserved unchanged
- How parsed staging tables will be created
- How file metadata should be captured
- How load timestamps should be tracked
- How schema changes should be detected
- How invalid records should be handled

Expected output:

- Raw layer design
- Staging table design
- SQL CREATE TABLE scripts or equivalent
- Explanation of design decisions

Question 3: JSON Parsing and Data Loading

Load and parse the six JSON files into structured tables.

Your answer should cover:

- JSON arrays or record-based JSON
- Date and timestamp conversion
- Numeric fields
- Text fields

- Missing fields
- Duplicate records
- Invalid records
- Source-to-target mapping

Expected output:

- Data loading scripts
- JSON parsing scripts
- Staging tables
- Notes on parsing assumptions

Question 4: Relational Data Model Design

Design a reporting-ready relational model from the JSON files. Your model should include fact and dimension tables.

Your answer should cover:

- Possible fact tables: fact_orders, fact_order_items and fact_deliveries
- Possible dimension tables: dim_customer, dim_store, dim_product, dim_channel, dim_payment_method, dim_promo and dim_date
- The grain of each fact table
- Primary keys and foreign keys
- Relationships between tables
- How facts connect to dimensions
- How to avoid double counting
- How the model supports BI dashboard reporting

Expected output:

- Data model diagram
- SQL scripts for facts and dimensions
- Explanation of grain and relationships

Question 5: Data Transformation Logic

Prepare the data for reporting by creating clean transformed tables or views.

Your answer should cover:

- Standard date fields
- Order week
- Order month
- Order day of week
- Order hour
- Promo flag
- Delivery SLA met flag
- Delay minutes
- Customer type
- New vs returning customer flag
- Items per order
- Average order value
- Product revenue
- Store, city and channel mappings

Expected output:

- SQL transformation scripts
- Cleaned reporting tables or views

- Explanation of transformation rules

Question 6: Data Quality Checks

Create automated data quality checks that would run after each data load. Include at least 10 checks.

Your answer should cover:

- Record count check by file
- Primary key uniqueness check
- Foreign key integrity check
- Null check on critical fields
- Duplicate order check
- Duplicate customer check
- Negative revenue check
- Invalid date check
- Missing store mapping check
- Missing product mapping check
- Missing delivery record for delivery orders
- Invalid delivery time check
- Promo code validation
- Revenue reconciliation check
- Order-item reconciliation check
- Unusual volume change check
- Late-arriving data check

Expected output:

- SQL queries for data quality checks
- Pass or fail result table
- Exception table showing failed records
- Explanation of what each check protects against

Question 7: Reconciliation and Audit Controls

Build reconciliation logic to prove that reporting outputs match the source data.

Your answer should cover:

- Total source records versus staging records
- Total orders from source versus reporting table
- Total revenue from source versus reporting table
- Store-level order and revenue reconciliation
- Channel-level order and revenue reconciliation
- Delivery record reconciliation
- Product revenue reconciliation
- Promo order reconciliation

Expected output:

- SQL reconciliation queries
- Reconciliation summary table
- Explanation of any variance found
- Recommended approach for audit logging

Question 8: Reporting Views and BI Semantic Layer

Create reporting-ready outputs that can be connected to a BI tool.

Your answer should cover:

- Final reporting tables or views
- Standard KPI definitions
- Fields exposed to the BI layer
- Calculations handled in SQL versus the BI tool
- Recommended Power BI or Tableau model structure
- Any row-level security considerations

Question 9: Pipeline Design and Automation

Design a daily automated pipeline for the JSON files.

Your answer should cover:

- How JSON files will arrive
- Where raw JSON files will be stored
- How files will be validated
- How data will be parsed
- How transformations will run
- How data quality checks will run
- How failed records will be handled
- How successful loads will be logged
- How the BI dashboard will be refreshed
- How late or corrected files will be handled

Question 10: API Integration Extension

Assume that in the future, the delivery partner will stop sending JSON files and will provide delivery data through an API. Design how you would ingest delivery data from the vendor API.

Your answer should cover:

- Authentication method
- Secure credential storage
- API endpoint design assumptions
- Pagination handling
- Rate-limit handling
- Incremental extraction
- Watermark logic
- Raw API response storage
- Deduplication
- Error handling
- Retry logic
- Monitoring and alerting
- Reconciliation against vendor totals
- Loading into the warehouse

Question 11: Monitoring and Incident Management

Design monitoring for the production BI data pipeline. Scenario: The dashboard refreshes successfully at 8:00 a.m., but leadership reports that some stores are missing from the sales report. Explain how you would investigate and resolve the issue.

Your answer should cover:

- Pipeline success or failure monitoring
- Data freshness checks
- Missing file checks
- Missing store data checks
- Row count anomaly detection
- Revenue anomaly detection
- Data quality failure alerts
- Dashboard refresh monitoring
- Incident logging
- Escalation process

Question 12: Technical Documentation

Prepare short technical documentation for your solution.

Your answer should cover:

- Source file description
- Source-to-target mapping
- Table dictionary
- Data model explanation
- Data flow explanation
- Transformation rules
- Data quality checks
- Reconciliation controls
- Refresh assumptions
- Known limitations
- Future improvement opportunities

6. Required Deliverables

1. SQL Scripts

Submit work covering the following:

- Table creation scripts
- Raw and staging table scripts
- JSON parsing scripts
- Data cleaning scripts
- Transformation scripts
- Data quality check queries
- Reconciliation queries
- Final reporting views or marts

Accepted file format: .sql, .txt, .ipynb or clearly documented notebook format.

2. JSON Loading and Parsing Documentation

Submit work covering the following:

- Tools used
- Parsing method
- Data type handling
- Error handling
- Assumptions
- Any challenges encountered

Accepted format: Word document, PDF, Markdown or notebook comments.

3. Data Model Diagram

Submit work covering the following:

- Fact tables
- Dimension tables
- Primary keys
- Foreign keys
- Relationships
- Cardinality, where possible
- Grain of each fact table

Accepted format: PDF, PowerPoint, image, Draw.io export, screenshot or BI model screenshot.

4. Data Flow Diagram

Submit work covering the following:

- JSON source files
- Raw layer
- Staging layer
- Transformation layer
- Warehouse or reporting layer
- BI semantic model or dashboard layer
- Data quality layer
- Error handling layer

Accepted format: PDF, PowerPoint, image, Draw.io export or screenshot.

5. Data Quality and Reconciliation Summary

Submit work covering the following:

- Checks performed
- Results of each check
- Exceptions found
- How exceptions were handled
- Risks or limitations

Accepted format: Excel, PDF, Word document, SQL output screenshots or notebook output.

6. API Integration Design

Submit work covering the following:

- Authentication
- Pagination
- Rate limits
- Incremental extraction
- Raw response storage

- Deduplication
- Error handling
- Monitoring
- Reconciliation
- Loading into the warehouse

Accepted format: PDF, Word document, PowerPoint slide or Markdown document.

7. Technical Documentation

Submit work covering the following:

- Source-to-target mapping
- Table dictionary
- Transformation rules
- KPI definitions
- Refresh assumptions
- Data quality rules
- Known limitations
- Future improvements

Accepted format: Word document, PDF, Markdown or PowerPoint.